

```
import os
import zipfile
import numpy as np
import matplotlib.pyplot as plt

from PIL import Image

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
from google.colab import files

uploaded = files.upload()
```

archive.zip
archive.zip(application/x-zip-compressed) - 2447903 bytes, last modified: 03/09/2026 - 100% done
 Saving archive.zip to archive.zip

```
zip_file = next(name for name in uploaded.keys() if name.endswith(".zip"))

with zipfile.ZipFile(zip_file, "r") as zip_ref:
    zip_ref.extractall("/content/apple_tomato")

print("Dataset extracted successfully!")
```

Dataset extracted successfully!

```
for root, dirs, files_list in os.walk("/content/apple_tomato"):
    print(root)
    print("Folders:", dirs)
    print("Files:", len(files_list))
    print()
```

```
/content/apple_tomato
Folders: ['test', 'train']
Files: 0
```

```
/content/apple_tomato/test
Folders: ['tomatoes', 'apples']
Files: 0
```

```
/content/apple_tomato/test/tomatoes
Folders: []
Files: 43
```

```
/content/apple_tomato/test/apples
Folders: []
Files: 54
```

```
/content/apple_tomato/train
Folders: ['tomatoes', 'apples']
Files: 0
```

```
/content/apple_tomato/train/tomatoes
Folders: []
Files: 130
```

```
/content/apple_tomato/train/apples
Folders: []
Files: 164
```

```
apple_path = "/content/apple_tomato/train/apples"
tomato_path = "/content/apple_tomato/train/tomatoes"

print("Apple images:", len(os.listdir(apple_path)))
print("Tomato images:", len(os.listdir(tomato_path)))
```

```
Apple images: 164
Tomato images: 130
```

```
apple_image = os.listdir(apple_path)[0]
tomato_image = os.listdir(tomato_path)[0]

apple_img = Image.open(os.path.join(apple_path, apple_image))
tomato_img = Image.open(os.path.join(tomato_path, tomato_image))

plt.figure(figsize=(8, 4))
```

```
plt.subplot(1, 2, 1)
plt.imshow(apple_img)
plt.title("Apple")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(tomato_img)
plt.title("Tomato")
plt.axis("off")

plt.show()
```

Apple



Tomato



```
IMG_SIZE = 64

X = []
y = []

# Apple = 0
for filename in os.listdir(apple_path):
    try:
        img_path = os.path.join(apple_path, filename)

        img = Image.open(img_path).convert("RGB")
        img = img.resize((IMG_SIZE, IMG_SIZE))

        img_array = np.array(img)

        X.append(img_array.flatten())
        y.append(0)

    except:
        pass

# Tomato = 1
for filename in os.listdir(tomato_path):
    try:
        img_path = os.path.join(tomato_path, filename)

        img = Image.open(img_path).convert("RGB")
        img = img.resize((IMG_SIZE, IMG_SIZE))

        img_array = np.array(img)

        X.append(img_array.flatten())
        y.append(1)

    except:
        pass

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (294, 12288)
y shape: (294,)
```

```
X = X / 255.0

print("Minimum pixel value:", X.min())
print("Maximum pixel value:", X.max())
```

```
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (235, 12288)
Testing data: (59, 12288)
```

```
model = LogisticRegression(
    max_iter=1000
)

model.fit(X_train, y_train)

print("Model training completed!")
```

```
Model training completed!
```

```
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Accuracy percentage:", accuracy * 100, "%")
```

```
Accuracy: 0.6271186440677966
Accuracy percentage: 62.71186440677966 %
```

```
cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[19 14]
 [ 8 18]]
```

```
plt.figure(figsize=(5, 4))

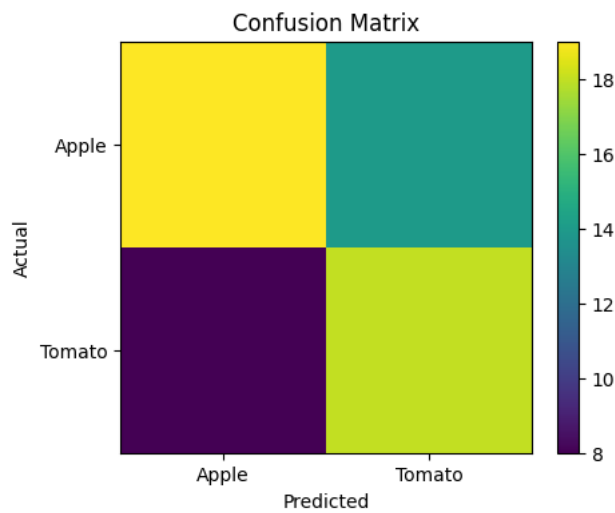
plt.imshow(cm)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.xticks([0, 1], ["Apple", "Tomato"])
plt.yticks([0, 1], ["Apple", "Tomato"])

plt.colorbar()

plt.show()
```



```
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["Apple", "Tomato"]
    )
)
```

	precision	recall	f1-score	support
Apple	0.70	0.58	0.63	33
Tomato	0.56	0.69	0.62	26
accuracy			0.63	59
macro avg	0.63	0.63	0.63	59
weighted avg	0.64	0.63	0.63	59

```
from google.colab import files
```

```
uploaded_image = files.upload()
```

apple2.jpg

apple2.jpg(image/jpeg) - 26800 bytes, last modified: 03/09/2026 - 100% done
Saving apple2.jpg to apple2.jpg

```
filename = list(uploaded_image.keys())[0]

img = Image.open(filename).convert("RGB")
img = img.resize((IMG_SIZE, IMG_SIZE))

img_array = np.array(img)
img_array = img_array.flatten()
img_array = img_array / 255.0

prediction = model.predict([img_array])[0]

probability = model.predict_proba([img_array])[0]

if prediction == 0:
    print("🍏 Prediction: APPLE")
    print("Confidence:", probability[0] * 100, "%")
else:
    print("🍅 Prediction: TOMATO")
    print("Confidence:", probability[1] * 100, "%")
```

🍏 Prediction: APPLE
Confidence: 97.8885068297496 %

```
print("Apple = 0")
print("Tomato = 1")

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nActual test labels:")
print(np.bincount(y_test))
```

```
nprint("\n\nPredicted test labels:")
```

```
Apple = 0  
Tomato = 1  
Training samples: 235  
Testing samples: 59
```

```
Actual test labels:  
[33 26]
```

```
Predicted test labels:  
[27 32]
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.6271186440677966
```

```
apple_test = os.listdir(apple_path)[10]
```

```
img = Image.open(  
    os.path.join(apple_path, apple_test)  
) .convert("RGB")
```

```
img = img.resize((IMG_SIZE, IMG_SIZE))
```

```
plt.imshow(img)  
plt.axis("off")  
plt.show()
```



```
img_array = np.array(img)  
img_array = img_array.flatten()  
img_array = img_array / 255.0
```

```
prediction = model.predict([img_array])[0]  
probability = model.predict_proba([img_array])[0]
```

```
print("Prediction:", prediction)  
print("Probabilities:", probability)
```

```
if prediction == 0:  
    print("🍏 APPLE")  
else:  
    print("🍅 TOMATO")
```

```
Prediction: 0  
Probabilities: [0.97034464 0.02965536]  
🍏 APPLE
```